

Examen

15 janvier 2019

Antoine Miné

Déroulement de l'examen

L'examen dure 2h.

Tous les documents papier sont autorisés (copie des transparents, notes de cours personnelles, livres, etc.). L'utilisation de tout moyen de communication (téléphone, tablette, etc.) est interdit. L'utilisation d'une clé USB ou disque USB personnel, en lecture seule et étiqueté à votre nom, est autorisée ; vous ne devez pas le prêter à un camarade.

Au début de l'examen, vous trouverez sur votre compte une copie des sources d'ILP dans le projet Eclipse `ILP-UPMC` du répertoire de travail d'Eclipse `workspace`. Lors de l'examen, vous complétez ce projet Eclipse en répondant aux questions de cet énoncé. Votre copie sera formée de fichiers textuels que vous laisserez aux endroits spécifiés dans votre espace de travail Eclipse. Afin de faciliter la correction, assurez-vous de placer les fichiers demandés dans le paquetage, donc dans le répertoire, précisé pour chaque question.

Le point de départ de l'examen est la version **ILP 4**. Tous vos fichiers doivent être rédigés dans le paquetage `com.paracampus.ilp4.exam1819` et ses sous-paquetages du projet Eclipse `ILP-UPMC`. Ils se trouveront donc dans un sous-répertoire de `workspace/ILP-UPMC/Java/src/com/paracampus/ilp4/exam1819` de votre compte.

L'examen sera corrigé à la main. Quand du code est demandé, les erreurs de syntaxe ne seront pas pénalisées. Il est souhaitable de n'expliciter que les parties du code qui vous semblent importantes, et de remplacer les parties peu informatives par un commentaire indiquant succinctement ce qui a été omis (c'est en particulier le cas du code répétitif, dont une version complète ne sera donnée qu'une seule fois).

L'examen est noté sur 22. Le barème pour chaque question n'est donné qu'à titre indicatif et pourra être adapté lors de la correction.

Mise en route

- Ouvrez Eclipse.
- Si demandé, choisissez le répertoire `workspace` comme espace de travail Eclipse. Attention, n'utilisez pas `eclipse-workspace`, parfois proposé à la place.
- Si le projet `ILP-UPMC` n'apparaît pas dans le « Package Explorer », ouvrez le menu « File / Open Projects From Filesystem... », cliquez sur « Directory... », choisissez le répertoire `workspace/ILP-UPMC`, puis « Finish ».
- Ouvrez le projet `ILP-UPMC` contenant les sources d'ILP4 et vérifiez qu'il compile bien.
- Le plug-in ANTLR devrait fonctionner sans réglage particulier. Vous pouvez toutefois générer les grammaires à la main avec un clic droit sur le fichier `.g4`, suivi de « Run as... / Generate ANTLR Recognizer ». Vérifiez que les fichiers Java générés sont dans le répertoire `workspace/ILP-UPMC/target/generated-sources/antlr4/antlr4` (le répertoire `antlr4` apparaît bien deux fois dans ce chemin) et comportent une directive `package antlr4`.

Vérifiez enfin que la liste des répertoires sources compilés par Eclipse (« Build Path ») comporte le répertoire `target/generated-sources/antlr4` (clic droit sur le projet, puis « Build Path / Configure Build Path... / onglet Source » ; le répertoire `antlr4` n'apparaît qu'une seule fois ici).

En cas de problème de compilation, vous pouvez modifier les options d'ANTLR avec un clic droit sur la grammaire, suivi de « Run As... / External Tools Configurations / Tools » et ajouter dans la zone « Arguments » l'option : « `-o HOME/workspace/ILP-UPMC/target/generated-sources/antlr4` » où `HOME` est remplacé par votre répertoire racine `/home/.../`.

Il est conseillé d'écrire des grammaires sans directive `import`, mais avec une directive `@header { package antlr4; }`, comme indiqué dans la correction du partiel.

- Vous pouvez également utiliser ANTLR en ligne de commande à l'aide de l'archive fournie dans les sources d'ILP, en faisant :

```
cd ~/workspace/ILP-UPMC/ANTLRGrammars
java -jar ~/workspace/ILP-UPMC/Java/jars/antlr-4.4-complete.jar \
  -lib ~/workspace/ILP-UPMC/ANTLRGrammars \
  -o ~/workspace/ILP-UPMC/target/generated-sources/antlr4/ \
  LE_NOM_DE_VOTRE_GRAMMAIRE.g4
```

- Ouvrez un navigateur et allez à l'URL : <http://www-master.ufr-info-p6.jussieu.fr/site-annuel-courant/DLP>. Vous y trouverez notamment les transparents du cours.
- Ouvrez également <https://www-ppti.ufr-info-p6.jussieu.fr>. Vous y trouverez une copie locale de nombreuses documentations, dont notamment celle de Java (menu Support, option Documentation).

Dépannage

Si Eclipse se bloque en essayant d'installer un connecteur SVN, tapez dans un terminal :

```
killall eclipse; killall java
```

puis relancez Eclipse.

Introduction

Dans cet examen, nous ajoutons une construction de type *try-with-resources*, comme introduite en Java 7. Il s'agit d'une construction qui déclare une ressource (fichier, processus, etc.) qui doit être libérée après utilisation, même en cas d'exception. En ILP, nous utiliserons la syntaxe suivante :

```
autoclose var = expr1 in expr2
```

où `autoclose` est un nouveau mot-clé du langage, `var` est une variable, et `expr1` et `expr2` sont des expressions.

Cette syntaxe, proche de `let var = expr1 in expr2`, aura la signification suivante :

- commencer par évaluer l'expression `expr1`, qui doit donner une instance *i* de classe ;
- évaluer ensuite l'expression `expr2` dans un environnement où la variable locale `var` est affectée à l'instance *i* ;
- si `expr2` s'évalue sans exception, la méthode `close()` est appelée sur l'instance *i*, puis la valeur de `expr2` est retournée ;
- si `expr2` signale une exception, la méthode `close()` est appelée sur l'instance *i*, puis l'exception est propagée au bloc englobant ;
- comme pour un `let var = expr1 in expr2`, la variable `var` est locale à `expr2` ;
- en cas d'exception dans `expr1`, celle-ci est propagée normalement, sans évaluer `expr2` ni appeler la méthode `close()` ;

- si *i* n'est pas une instance d'une classe qui possède une méthode `close()` sans argument, une erreur doit être signalée.

Voici un exemple plus complet :

```
1 class File {
2     var isOpen;
3     method close() ( self.isOpen = false )
4     method prn(v) ( if self.isOpen then print(v) else print ("CLOSED") )
5 }
6
7 autoclose f = new File(true) in ( g = f; f.prn("mot1 ") );
8 g.prn("mot2");
```

Son exécution affichera : « mot1 CLOSED ». En effet, le premier appel à `prn` se fait alors que le fichier `f` est ouvert (`isOpen` vaut `true` à la création de `f`), alors que le second appel se fait après l'exécution de `autoclose`, donc après l'appel implicite à `close()`, et donc avec `isOpen` valant `false`. Notez que la variable globale `g` pointe sur la même instance que `f`, mais reste visible après l'exécution de l'instruction `autoclose`, au delà de la portée de la variable locale `f`.

1 Grammaire (2 points)

Écrivez une grammaire qui étend ILP4 avec cette nouvelle construction.

Cette grammaire se trouvera dans le fichier `workspace/ILP-UPMC/Java/src/com/paracampus/ilp4/exam1819/grammar/ILPgrammar4exam1819.g4`.

2 AST (3 points)

Proposez une extension de l'arbre syntaxique d'ILP4 avec cette nouvelle construction.

Vous devez fournir toutes les classes AST dans le paquetage `com.paracampus.ilp4.exam1819.ast`, mais aussi les interfaces IAST dans le paquetage `com.paracampus.ilp4.exam1819.interfaces`.

Vous fournirez également les classes et interfaces permettant d'étendre la fabrique d'AST et le visiteur d'AST.

3 Analyse syntaxique (2 points)

Écrivez dans le paquetage `com.paracampus.ilp4.exam1819.parse` les nouvelles classes `ILPMLparser` et `ILPMLlistener` permettant de construire les nœuds ajoutés au langage.

4 Réécriture d'AST (4 points)

Notre nouvelle construction `autoclose var = expr1 in expr2` peut être réécrite à l'aide des constructions existantes `let ... in ...` et `try ...catch ... finally ...`.

1. Transformez l'exemple ci-dessus (utilisant la classe `File`) en un programme ILP4 équivalent sans construction `autoclose`.

Vous fournirez votre réponse dans le fichier texte : `workspace/ILP-UPMC/Java/src/com/paracampus/ilp4/exam1819/ast/exemple.txt`.

2. Écrivez un visiteur `com.paracampus.ilp4.exam1819.ast.AutoCloseTransform` capable de transformer un AST ILP4 contenant des constructions `autoclose` en un AST ILP4 équivalent ne contenant aucun `autoclose`.

Nous ne demandons pas le code complet de cette classe : vous fournirez les méthodes qui vous semblent les plus importantes et vous pourrez omettre les méthodes répétitives,

à condition de préciser informellement mais clairement (par exemple sous forme de commentaires dans le fichier Java demandé) la liste complète des méthodes que vous devriez implanter et leur fonctionnement général.

5 Interprétation (4 points)

Dans la suite de l'examen, nous repartons d'un AST ILP4 enrichi pouvant contenir la construction `autoclose`. Nous n'utilisons donc pas le résultat de la question précédente.

Implantez un nouvel interprète dans une classe `com.paracamplus.ilp4.exam1819.interpreter.Interpreter` pour les programmes ILP4 avec `autoclose`.

6 Compilation (5 points)

Nous nous intéressons maintenant à la compilation de programmes ILP4 avec `autoclose`.

1. Donnez la liste de toutes les interfaces et les classes que vous auriez à ajouter pour implanter le compilateur. Précisez leur nom et le paquetage complet. Pour chaque interface, indiquez les interfaces dont elle hérite. Pour chaque classe, indiquez la classe dont elle hérite et les interfaces qu'elle implante. On ne demande pas le code Java de ces classes et interfaces.
Vous fournirez votre réponse dans un fichier texte : `workspace/ILP-UPMC/Java/src/com/paracamplus/ilp4/exam1819/compiler/classes.txt`.
2. Donnez le code de la méthode du visiteur de normalisation, `Normalizer`, qui traite le nœud `autoclose` (on ne demande pas les autres méthodes).
Vous fournirez votre réponse dans le fichier : `workspace/ILP-UPMC/Java/src/com/paracamplus/ilp4/exam1819/compiler/normalizer/Normalizer.java`.
3. Donnez le code de la méthode du visiteur d'analyse des variables libres, `FreeVariableCollector`, qui traite le nœud `autoclose` (on ne demande pas les autres méthodes).
Vous fournirez votre réponse dans le fichier : `workspace/ILP-UPMC/Java/src/com/paracamplus/ilp4/exam1819/compiler/FreeVariableCollector.java`.
4. Donnez un schéma de compilation détaillé pour la construction `autoclose`.
Vous fournirez votre réponse dans un fichier texte : `workspace/ILP-UPMC/Java/src/com/paracamplus/ilp4/exam1819/compiler/schema.txt`. On ne demande pas le code Java de la classe `Compiler`.
5. Indiquez les fonctions de la bibliothèque d'exécution C que vous utilisez pour implanter `autoclose`. Expliquez leur utilité et leur fonctionnement. Précisez si vous devez enrichir la bibliothèque d'exécution.
Vous fournirez votre réponse dans un fichier texte : `workspace/ILP-UPMC/Java/src/com/paracamplus/ilp4/exam1819/compiler/lib.txt`.

7 Tests (2 points)

Dans un répertoire `workspace/ILP-UPMC/Java/src/com/paracamplus/ilp4/exam1819/tests/`, fournissez des tests de la construction `autoclose`.

Pour chaque test, vous fournirez le source ILP4, mais aussi les fichiers permettant de vérifier les résultats attendus. Vous indiquerez brièvement en commentaire dans chaque fichier source ILP4 le comportement qui est testé par le fichier.

Vous prendrez garde à fournir suffisamment de tests pour couvrir l'ensemble des comportements possibles.